Quick Reference Handbook

DEVOPS ENGINEERING MASTERY — Volume 10, Companion Volume. First Edition, 9 November 2026. Bengali-English glossary (180+ terms), Prometheus, PromQL, Grafana, LogQL and OpenTelemetry references, Kubernetes monitoring commands, incident investigation checklist, SLI/SLO reference, checklists, troubleshooting decision trees, the competency tracking sheet and official references — keep this beside your monitoring lab. Prometheus, Grafana, Loki, OpenTelemetry, Node.js 20.

Quick Reference Handbook • Bengali–English Glossary • 1–60

Bengali–English Glossary (1–60)

প্রতিটি টার্মের বাংলা ব্যাখ্যার সঙ্গে বাস্তব ব্যবহারের প্রসঙ্গ দেওয়া হয়েছে। Technical term ইংরেজিতে রাখা হয়েছে।

Observability: সিস্টেমের ভেতরের অবস্থা বাইরের ডেটা (metrics, logs, traces) দিয়ে বোঝার ক্ষমতা। প্রোডাকশনে "কেন ধীর?" প্রশ্নের উত্তর দিতে এটি লাগে।

Monitoring: আগে থেকে ঠিক করা বিষয় মাপা ও threshold ছাড়ালে alert দেওয়া। যেমন CPU 90% ছাড়ালে alert।

Telemetry: সিস্টেম থেকে সংগ্রহ করা পর্যবেক্ষণ ডেটা — metrics, logs ও traces। এটি observability-র কাঁচামাল।

Metrics: সময়ের সঙ্গে পরিবর্তিত সংখ্যাগত পরিমাপ, যেমন request সংখ্যা বা latency। Prometheus-এ সংরক্ষণ করা হয়।

Logs: সিস্টেমে ঘটে যাওয়া ঘটনার সময়সহ বিবরণ। Loki বা Elasticsearch-এ সংরক্ষণ করা হয়।

Traces: একটি request বিভিন্ন সার্ভিসের মধ্য দিয়ে যাওয়ার পথ ও সময়ের বিবরণ। Tempo বা Jaeger-এ দেখা যায়।

Span: একটি trace-এর মধ্যে একটি কাজের সময়কাল, যেমন HTTP request বা DB query। প্রতিটি span-এর শুরু, শেষ ও নাম থাকে।

Trace ID: একটি request-এর সব span-কে যুক্ত করার একক শনাক্তকারী। Log-এ trace_id রাখলে log ও trace যুক্ত করা যায়।

Counter: শুধু বাড়ে এমন metric, যেমন মোট request সংখ্যা। Restart-এ শূন্য হয়।

Gauge: বাড়ে-কমে এমন metric, যেমন সক্রিয় সংযোগের সংখ্যা বা বর্তমান memory।

Histogram: মানের বণ্টন bucket-এ ধরে রাখা metric, যেমন latency। এতে p95/p99 বের করা যায়।

Summary: ক্লায়েন্টে percentile হিসাব করে রাখা metric। Aggregate করা যায় না বলে Prometheus-এ কম ব্যবহার করা হয়।

Bucket: Histogram-এ মানের সীমা, যেমন 0.1s, 0.5s, 1s। Bucket নির্বাচন percentile-এর নির্ভুলতা ঠিক করে।

Percentile (p95/p99): 95% বা 99% request-এর latency-র নিচে থাকা মান। গড়ের চেয়ে ব্যবহারকারীর অভিজ্ঞতা ভালো বোঝায়।

Label: Metric-এর সঙ্গে যুক্ত key-value তথ্য, যেমন `service="api"`। Label দিয়ে filter ও group করা যায়।

Cardinality: একটি metric-এ লেবেলের সম্ভাব্য মানের সংখ্যা। বেশি cardinality storage ও memory চাপ বাড়ায়।

Series (Time Series): একটি metric নাম ও লেবেলের সমন্বয়ে সময়ের সঙ্গে ধারাবাহিক মান। প্রতিটি series আলাদাভাবে সংরক্ষণ হয়।

Scrape: Prometheus-এর target থেকে metrics টেনে আনা। সাধারণত 15s বা 30s পরপর হয়।

Scrape Interval: দুটি scrape-এর মধ্যে সময়। খুব ছোট interval চাপ বাড়ায়, খুব বড় interval বিস্তারিত কমায়।

Target: Prometheus যে endpoint থেকে metrics সংগ্রহ করে, যেমন `api:3000/metrics`।

Exporter: কোনো সিস্টেমের metrics Prometheus ফরম্যাটে প্রকাশ করা প্রোগ্রাম। Node Exporter এর উদাহরণ।

Node Exporter: হোস্ট মেশিনের CPU, memory, disk ও network metrics দেওয়ার exporter।

Pushgateway: স্বল্পস্থায়ী job-এর metrics Prometheus-এ পাঠানোর মধ্যবর্তী সার্ভিস। দীর্ঘস্থায়ী সার্ভিসে ব্যবহার করা উচিত নয়।

Service Discovery: নতুন target স্বয়ংক্রিয়ভাবে খুঁজে বের করার ব্যবস্থা। Kubernetes-এ Pod IP বদলালেও কাজ করে।

PromQL: Prometheus-এর query ভাষা। Metrics বেছে নেওয়া, aggregate করা ও alert-এর শর্ত লেখা যায়।

Rate: PromQL ফাংশন যা Counter থেকে প্রতি সেকেন্ডে হার বের করে। যেমন `rate(http_requests_total[5m])`।

Increase: PromQL ফাংশন যা নির্দিষ্ট সময়ে Counter-এর মোট বৃদ্ধি দেয়।

Recording Rule: ভারী query-র ফলাফল আগে থেকে হিসাব করে নতুন metric হিসেবে সংরক্ষণ করার নিয়ম। Dashboard দ্রুত করে।

Alerting Rule: PromQL শর্ত সত্য হলে alert তৈরি করার নিয়ম। `for` সময় দিয়ে ক্ষণিকের spike এড়ানো যায়।

Alertmanager: Alert গ্রহণ করে group, deduplicate, silence ও সঠিক receiver-এ পাঠানোর সার্ভিস।

Grouping: একই ধরনের alert একসঙ্গে পাঠানো। Alert storm কমায়।

Deduplication: একই alert বারবার পাঠানো আটকানো। Replica Prometheus থাকলে এটি জরুরি।

Silence: নির্দিষ্ট সময়ের জন্য alert চুপ করানো, যেমন রক্ষণাবেক্ষণের সময়। Silence-এর মেয়াদ ও কারণ লিখতে হয়।

Inhibition: একটি alert সক্রিয় থাকলে সম্পর্কিত কম গুরুত্বপূর্ণ alert চাপা দেওয়া। যেমন সার্ভিস down থাকলে তার latency alert চাপা দেওয়া।

Receiver: Alert পাঠানোর গন্তব্য, যেমন Slack, ইমেইল বা PagerDuty। Routing দিয়ে ঠিক করা হয়।

Routing: Alert-এর লেবেল অনুযায়ী কোন receiver-এ পাঠানো হবে তা ঠিক করা। Severity অনুযায়ী routing করা যায়।

Severity: Alert-এর গুরুত্ব, যেমন critical, warning, info। Critical মানে এখনই ব্যবস্থা নিতে হবে।

Alert Fatigue: অনেক অপ্রয়োজনীয় alert আসায় গুরুত্বপূর্ণ alert উপেক্ষিত হওয়া। Actionable alert রেখে এটি কমানো যায়।

Actionable Alert: যে alert পেয়ে কী করতে হবে তা স্পষ্ট থাকে। Runbook link থাকলে আরও ভালো হয়।

Dashboard: Metrics ও log visualize করার প্যানেলের সমষ্টি। Grafana-তে তৈরি করা হয়।

Panel: Dashboard-এর একটি অংশ, যেমন একটি graph বা table। প্রতিটি panel একটি query চালায়।

Variable: Grafana dashboard-এ পুনঃব্যবহারযোগ্য মান, যেমন `$service`। একই dashboard একাধিক service-এর জন্য ব্যবহার করা যায়।

Data Source: Grafana যেখান থেকে ডেটা আনে, যেমন Prometheus, Loki বা Tempo। Provisioning YAML দিয়ে সংজ্ঞায়িত করা যায়।

Provisioning: কোড (YAML) দিয়ে Grafana-র data source ও dashboard সংজ্ঞায়িত করা। পুনরুৎপাদনযোগ্য সেটআপ দেয়।

SLI (Service Level Indicator): সার্ভিসের নির্ভরযোগ্যতার পরিমাপযোগ্য নির্দেশক, যেমন সফল request-এর অনুপাত।

SLO (Service Level Objective): SLI-র লক্ষ্য মান, যেমন 99.9% সফল request। এটি অভ্যন্তরীণ লক্ষ্য।

SLA (Service Level Agreement): গ্রাহকের সঙ্গে চুক্তিতে লেখা নির্ভরযোগ্যতার প্রতিশ্রুতি। ভাঙলে ক্ষতিপূরণ থাকতে পারে।

Error Budget: 100% থেকে SLO বিয়োগ করে পাওয়া অনুমোদিত ব্যর্থতা। বাজেট শেষ হলে release থামানো হয়।

Burn Rate: Error budget কত দ্রুত শেষ হচ্ছে তার হার। দ্রুত burn মানে জরুরি পদক্ষেপ দরকার।

Golden Signals: Latency, Traffic, Errors, Saturation — সার্ভিস মূল্যায়নের চারটি মূল সংকেত।

Latency: একটি request সম্পন্ন হতে কত সময় লাগে। p95/p99 দিয়ে মাপা হয়।

Traffic: সার্ভিসে কত request আসছে তার হার। চাহিদা বোঝায়।

Errors: ব্যর্থ request-এর হার। 5xx error-এর অনুপাত দেখা হয়।

Saturation: কোনো resource কতটা পূর্ণ বা ব্যস্ত। CPU 95% বা queue ভর্তি হওয়া এর উদাহরণ।

RED Method: Rate, Errors, Duration দিয়ে request-driven সার্ভিস মূল্যায়ন করার পদ্ধতি। API-র জন্য ব্যবহার করা হয়।

USE Method: Utilization, Saturation, Errors দিয়ে resource মূল্যায়ন করার পদ্ধতি। CPU, disk, network-এর জন্য।

Availability: সার্ভিস কত সময় ব্যবহারযোগ্য ছিল তার অনুপাত। শুধু availability মাপা যথেষ্ট নয়, latency-ও মাপতে হয়।

Uptime: সার্ভিস চালু থাকার সময়। Downtime-এর বিপরীত।

Downtime: সার্ভিস ব্যবহার করা যায়নি এমন সময়। SLO-তে এর সীমা থাকে।

Alerting: নির্দিষ্ট শর্ত সত্য হলে স্বয়ংক্রিয়ভাবে সতর্কবার্তা পাঠানো। ভালো alerting শুধু গুরুত্বপূর্ণ ও actionable alert পাঠায়।

Quick Reference Handbook • Bengali–English Glossary • 61–120

Bengali–English Glossary (61–120)

Structured Logging: Log-কে JSON-এর মতো field-সহ লেখা। level, trace_id, request_id আলাদা করে query করা যায়।

Unstructured Logging: সাধারণ বাক্যে লেখা log। পড়তে সহজ, কিন্তু নির্ভুলভাবে query করা কঠিন।

Log Level: Log-এর গুরুত্ব, যেমন DEBUG, INFO, WARN, ERROR। Production-এ DEBUG বন্ধ রাখা হয়।

LogQL: Loki-র query ভাষা। Label filter, line filter ও parser দিয়ে log খোঁজা যায়।

Label Filter: LogQL-এ `{app="api"}` -এর মতো stream বাছাই করা। এটি সবচেয়ে সস্তা filter।

Line Filter: LogQL-এ `|= "error"` -এর মতো লাইনের মধ্যে শব্দ খোঁজা। Label filter-এর পর প্রয়োগ হয়।

Parser: LogQL-এ log লাইন থেকে field বের করা, যেমন `| json` বা `| logfmt`। Structured log-এ parser কার্যকর।

Loki: Label-ভিত্তিক log সংগ্রহ ও query করার সিস্টেম। পুরো log index করে না বলে হালকা।

Grafana Alloy: Metrics, logs ও traces সংগ্রহ করে Prometheus/Loki/Tempo-তে পাঠানোর agent।

Elasticsearch: Log ও document-এর full-text search ও বিশ্লেষণের সিস্টেম। পুরো text index করে।

OpenSearch: Elasticsearch-এর community-driven fork। Search ও observability-র জন্য ব্যবহৃত হয়।

Kibana: Elasticsearch-এর ডেটা visualize করার টুল। OpenSearch-এ এর সমতুল্য OpenSearch Dashboards।

Mapping: Elasticsearch-এ field-এর ধরন (string, number, date) ঠিক করা। ভুল mapping-এ query ভেঙে যায়।

Index: Elasticsearch-এ search-এর জন্য সংগঠিত ডেটার একক। Log সাধারণত দিন অনুযায়ী index-এ রাখা হয়।

Shard: Elasticsearch index-এর একটি অংশ। বেশি shard search দ্রুত করে, কিন্তু resource বেশি লাগে।

OpenTelemetry (OTel): Metrics, logs ও traces সংগ্রহের vendor-neutral standard। SDK, API ও Collector নিয়ে গঠিত।

OTel Collector: Telemetry ডেটা গ্রহণ, প্রক্রিয়া ও পাঠানোর সার্ভিস। Receiver, Processor ও Exporter নিয়ে গঠিত।

Receiver: OTel Collector-এ ডেটা গ্রহণ করার অংশ। যেমন OTLP receiver।

Processor: OTel Collector-এ ডেটা রূপান্তর, filter বা sampling করার অংশ। যেমন batch processor।

Exporter: OTel Collector-এ ডেটা পাঠানোর অংশ। যেমন Prometheus, Loki বা Tempo exporter।

Pipeline: OTel Collector-এ receiver → processor → exporter-এর ক্রম। Metrics, logs ও traces-এর জন্য আলাদা pipeline থাকে।

Instrumentation: অ্যাপে metrics, logs ও traces সংগ্রহের কোড যোগ করা। Auto ও manual দুই ধরনের হয়।

Auto-Instrumentation: কোড না বদলে library দিয়ে স্বয়ংক্রিয়ভাবে span তৈরি করা। সাধারণ HTTP/DB span পাওয়া যায়।

Manual Instrumentation: নিজে কোডে span ও metric যোগ করা। ব্যবসায়িক যুক্তি মাপতে এটি দরকার।

Context Propagation: Trace ID ও span ID এক সার্ভিস থেকে অন্য সার্ভিসে header-এ পাঠানো। ভেঙে গেলে trace ভেঙে যায়।

Traceparent Header: W3C standard header যা trace context বহন করে। প্রতিটি সার্ভিস এটি forward করতে হয়।

Sampling: সব trace না রেখে একটি অংশ রাখা। খরচ কমায়, কিন্তু বিরল error হারাতে পারে।

Head Sampling: Trace শুরুতেই রাখবে কি না ঠিক করা। সস্তা, কিন্তু গুরুত্বপূর্ণ trace হারাতে পারে।

Tail Sampling: পুরো trace দেখে রাখবে কি না ঠিক করা। Error trace ধরে রাখতে পারে, কিন্তু বেশি resource লাগে।

Jaeger: Distributed trace visualize ও query করার টুল। Tempo-র সঙ্গে একসঙ্গে ব্যবহার করা যায়।

Tempo: Trace সংরক্ষণের ব্যাকএন্ড। Grafana-তে trace দেখা যায়।

APM: Application Performance Monitoring — অ্যাপের ভেতরের ধীর function, DB query ও error চিহ্নিত করা।

Profiling: কোন function বা allocation বেশি CPU/memory খাচ্ছে তা মাপা। Memory leak খুঁজতে কাজে আসে।

Heap Profile: Memory-তে কোন object কত জায়গা নিচ্ছে তার বিবরণ। Leak খুঁজতে ব্যবহার করা হয়।

Flame Graph: CPU বা memory ব্যবহার গাছের মতো দেখানো চিত্র। কোন function বেশি সময় নিচ্ছে তা বোঝা যায়।

Incident: সার্ভিসের উপর প্রভাব ফেলা অপ্রত্যাশিত ঘটনা। যেমন API down বা error spike।

Incident Response: Incident-এর প্রভাব কমানো, যোগাযোগ করা ও সমাধান করার প্রক্রিয়া। প্রথমে mitigation, তারপর root cause।

Mitigation: Incident-এর প্রভাব কমানো, যেমন rollback বা traffic সরানো। Root cause ঠিক করা নয়, ক্ষতি কমানো।

Root Cause: Incident-এর মূল কারণ। লক্ষণ নয়, যে ভুলটি ঠিক করলে সমস্যা আর ঘটবে না।

Postmortem: Incident-এর পর লেখা পর্যালোচনা — timeline, কারণ, শিক্ষা ও পদক্ষেপ। Blameless হওয়া উচিত।

Blameless Postmortem: ব্যক্তিকে দোষ না দিয়ে সিস্টেমের দুর্বলতা খোঁজা। মানুষ সত্য তথ্য দেয় ও স্থায়ী সমাধান হয়।

Runbook: জানা সমস্যার জন্য ধাপে ধাপে নির্ণয় ও সমাধানের নির্দেশনা। Alert-এ runbook link থাকা ভালো।

On-Call: নির্দিষ্ট সময়ে incident-এর দায়িত্বে থাকা ইঞ্জিনিয়ার। Rotation দিয়ে দায়িত্ব ভাগ করা হয়।

On-Call Rotation: দায়িত্ব পালাক্রমে ভাগ করা। ক্লান্তি কমায় ও সবসময় কেউ দায়িত্বে থাকে।

Pager: Critical alert-এ on-call-কে ডাকার ব্যবস্থা। যেমন PagerDuty বা Opsgenie।

Mean Time to Detect (MTTD): সমস্যা শুরু থেকে ধরা পড়া পর্যন্ত গড় সময়। ভালো monitoring MTTD কমায়।

Mean Time to Resolve (MTTR): সমস্যা ধরা পড়া থেকে সমাধান পর্যন্ত গড় সময়। Runbook ও প্রমাণ MTTR কমায়।

Health Check: অ্যাপ চালু ও কাজ করার মতো অবস্থায় আছে কি না তা জানানোর endpoint, যেমন `/health`।

Readiness Probe: Kubernetes-এ Pod traffic নেওয়ার জন্য প্রস্তুত কি না তা পরীক্ষা করা। প্রস্তুত না হলে traffic যায় না।

Liveness Probe: Kubernetes-এ Pod জীবিত আছে কি না তা পরীক্ষা করা। ব্যর্থ হলে Pod restart হয়।

Blackbox Monitoring: বাইরে থেকে endpoint পরীক্ষা করা, যেমন HTTP 200, TLS মেয়াদ, DNS। Blackbox exporter ব্যবহার করা হয়।

Whitebox Monitoring: সিস্টেমের ভেতরের metrics দেখে পরীক্ষা করা, যেমন Prometheus metrics।

Synthetic Monitoring: কৃত্রিম request পাঠিয়ে বাইরে থেকে সার্ভিস পরীক্ষা করা। বাস্তব ব্যবহারকারী ছাড়াই চলে।

Real User Monitoring (RUM): বাস্তব ব্যবহারকারীর ব্রাউজার থেকে অভিজ্ঞতা মাপা। আসল latency ও error দেখায়।

Retention: ডেটা কতদিন রাখা হবে তার নীতি। তারপর ডেটা মুছে ফেলা হয়।

Downsampling: পুরোনো ডেটার resolution কমিয়ে সংরক্ষণ করা। Storage বাঁচায়, বিস্তারিত কমায়।

High Availability (HA): একটি অংশ down হলেও সার্ভিস চলতে থাকা। Replica ও redundancy দিয়ে করা হয়।

Federation: একাধিক Prometheus থেকে নির্বাচিত metrics কেন্দ্রীয় Prometheus-এ আনা। Global দৃশ্যের জন্য।

Remote Write: Prometheus থেকে metrics দীর্ঘমেয়াদি storage-এ পাঠানো। Thanos বা Mimir-এ ব্যবহার করা হয়।

Thanos / Cortex / Mimir: Prometheus-এর দীর্ঘমেয়াদি storage, HA ও global query-র সমাধান।

Quick Reference Handbook • Bengali–English Glossary • 121–180

Bengali–English Glossary (121–180)

kube-state-metrics: Kubernetes object-এর অবস্থা (desired vs actual) metrics হিসেবে দেওয়ার সার্ভিস।

metrics-server: Kubernetes-এ Node ও Pod-এর বর্তমান CPU/memory ব্যবহার দেওয়ার সার্ভিস। `kubectl top` এটি ব্যবহার করে।

DaemonSet: Kubernetes-এ প্রতিটি node-এ একটি Pod চালানোর নিয়ম। Node Exporter DaemonSet হিসেবে চলে।

OOMKilled: Container memory limit ছাড়ালে kernel-এর মেরে ফেলা। Pod restart হয়।

CrashLoopBackOff: Pod বারবার crash করে restart হওয়ার অবস্থা। Log ও event-এ কারণ থাকে।

Connection Pool: ডেটাবেস সংযোগের পুনঃব্যবহারযোগ্য সংগ্রহ। Pool পূর্ণ হলে request অপেক্ষা করে।

Timeout: কোনো কাজের জন্য সর্বোচ্চ অপেক্ষার সময়। Timeout ছাড়ালে request ব্যর্থ হয়।

Retry: ব্যর্থ request আবার পাঠানো। বেশি retry downstream-কে চাপ দেয়, তাই backoff দরকার।

Retry Storm: অনেক retry একসঙ্গে এসে downstream-কে আরও চাপ দেওয়া। Circuit breaker এটি আটকায়।

Circuit Breaker: Downstream বারবার ব্যর্থ হলে সাময়িকভাবে request বন্ধ রাখা। Downstream-কে সুস্থ হওয়ার সময় দেয়।

Backoff: Retry-এর মধ্যে ক্রমবর্ধমান বিরতি। Exponential backoff চাপ কমায়।

Load Balancing: Traffic একাধিক সার্ভারে ভাগ করা। একটি Pod-এ চাপ কেন্দ্রীভূত হওয়া আটকায়।

Autoscaling: চাহিদা অনুযায়ী Pod বা node সংখ্যা স্বয়ংক্রিয়ভাবে বাড়ানো-কমানো। HPA এর উদাহরণ।

Horizontal Pod Autoscaler (HPA): CPU বা metrics অনুযায়ী Pod সংখ্যা বাড়ানো-কমানো।

Resource Limit: Container-এর সর্বোচ্চ CPU/memory। ছাড়ালে throttle বা OOMKilled হয়।

Resource Request: Container-এর জন্য সংরক্ষিত CPU/memory। Scheduler এটি দেখে Pod বসায়।

Throttle: CPU limit ছাড়ালে container-এর CPU সীমিত করা। Latency বাড়তে পারে।

Queue: কাজের অপেক্ষমাণ সারি। Queue ভর্তি হলে saturation বোঝায়।

Throughput: নির্দিষ্ট সময়ে সম্পন্ন কাজের সংখ্যা। যেমন প্রতি সেকেন্ডে request।

Error Rate: মোট request-এর মধ্যে ব্যর্থ request-এর অনুপাত। SLO-র মূল SLI।

5xx Error: সার্ভারের দিকের ত্রুটি, যেমন 500, 502, 503। সার্ভিসের সমস্যা বোঝায়।

4xx Error: ক্লায়েন্টের দিকের ত্রুটি, যেমন 404, 400। সাধারণত ব্যবহারকারীর request-এ সমস্যা।

Status Code: HTTP response-এর সংখ্যা যা ফলাফল বোঝায়। 2xx সফল, 4xx ক্লায়েন্ট ভুল, 5xx সার্ভার ভুল।

Endpoint: API-র একটি ঠিকানা, যেমন `/api/orders`। প্রতিটি endpoint-এর latency আলাদা মাপা যায়।

Dependency: একটি সার্ভিস যার উপর নির্ভর করে, যেমন ডেটাবেস বা payment gateway। Downstream failure-এর উৎস।

Downstream: একটি সার্ভিস যাকে ডাকা হয়। Downstream ধীর হলে নিজের সার্ভিসও ধীর হয়।

Upstream: একটি সার্ভিস যাকে ডাকে। Upstream-এর চাপ downstream-এ পড়ে।

Microservices: ছোট ছোট স্বাধীন সার্ভিসে ভাগ করা আর্কিটেকচার। Trace দিয়ে request-এর পথ বোঝা যায়।

Correlation: Metrics, logs ও traces-কে একই request বা ঘটনার সঙ্গে যুক্ত করা। trace_id দিয়ে করা হয়।

Single Pane of Glass: এক জায়গায় metrics, logs ও traces একসঙ্গে দেখা। Grafana-তে তিনটি data source যুক্ত করে করা যায়।

PII (Personally Identifiable Information): ব্যক্তিকে শনাক্ত করা যায় এমন তথ্য, যেমন নাম, ইমেইল, ফোন। Log-এ রাখা বিপজ্জনক।

Redaction: Log বা trace থেকে সংবেদনশীল তথ্য মুছে বা mask করে দেওয়া। Secret ফাঁস রোধ করে।

Access Control: কে কোন ডেটা দেখতে পারবে তা নিয়ন্ত্রণ করা। Role অনুযায়ী Grafana ও Loki-তে সীমা দেওয়া হয়।

Role-Based Access Control (RBAC): ভূমিকা (role) অনুযায়ী অনুমতি দেওয়া। যেমন viewer শুধু দেখতে পারে, editor বদলাতে পারে।

TLS Certificate: HTTPS-এর জন্য নিরাপত্তা সার্টিফিকেট। মেয়াদ শেষ হলে সাইট বন্ধ হয়, তাই expiry alert দরকার।

Certificate Expiry: TLS certificate-এর মেয়াদ শেষ হওয়া। Blackbox exporter দিয়ে 14 দিন আগে alert দেওয়া যায়।

Change Management: Deploy ও config পরিবর্তন পরিচালনা করা। প্রতিটি change-এর সময় লিপিবদ্ধ থাকলে incident-এর সঙ্গে মেলানো যায়।

Rollback: নতুন সংস্করণে সমস্যা হলে আগের সংস্করণে ফিরে যাওয়া। দ্রুত mitigation-এর উপায়।

Deploy: নতুন কোড বা config প্রোডাকশনে পাঠানো। Deploy-এর পর error ও latency নজরে রাখতে হয়।

Canary Deploy: প্রথমে অল্প ব্যবহারকারীর উপর নতুন সংস্করণ পরীক্ষা করা। সমস্যা হলে প্রভাব কম থাকে।

Blue-Green Deploy: নতুন সংস্করণ আলাদা পরিবেশে চালিয়ে traffic সরানো। Rollback দ্রুত করা যায়।

Feature Flag: কোড deploy না করে ফিচার চালু/বন্ধ করার সুইচ। সমস্যা হলে ফিচার বন্ধ করা যায়।

Capacity Planning: ভবিষ্যতের চাহিদা অনুযায়ী resource পরিকল্পনা করা। Metrics-এর trend দেখে করা হয়।

Cost Optimisation: Observability খরচ নিয়ন্ত্রণ করা — cardinality, sampling ও retention ঠিক করে।

Data Volume: সংগ্রহ করা telemetry ডেটার পরিমাণ। বেশি volume খরচ বাড়ায়।

Ingestion: Telemetry ডেটা গ্রহণ করা। Loki বা Prometheus-এ ingestion হার খরচ নির্ধারণ করে।

Query Performance: Query কত দ্রুত ফল দেয়। ভারী query recording rule দিয়ে দ্রুত করা যায়।

Time Range: Query-তে কোন সময়ের ডেটা দেখা হবে। বড় range ধীর হয়।

Aggregation: অনেক মানকে একত্র করে একটি মান বের করা, যেমন `sum` বা `avg`। PromQL-এ `by` দিয়ে group করা যায়।

Grouping (PromQL): `by` বা `without` দিয়ে series-কে ভাগ করা। যেমন `sum by (service)`।

Instant Vector: একটি নির্দিষ্ট সময়ের মানের সেট। যেমন `up`।

Range Vector: একটি সময়ের মধ্যে মানের সেট। যেমন `http_requests_total[5m]`। rate() range vector নেয়।

Histogram Quantile: Histogram থেকে percentile বের করার PromQL ফাংশন। `by (le)` দিয়ে aggregate করতে হয়।

Absent: কোনো metric একেবারেই আসছে না তা ধরার PromQL ফাংশন। Job বন্ধ হলে alert দিতে ব্যবহার করা হয়।

Topk: সবচেয়ে বড় মানের series বেছে নেওয়ার PromQL ফাংশন। সময়ের সঙ্গে series বদলায় বলে সতর্কভাবে ব্যবহার করুন।

Promtool: Prometheus-এর config ও rules যাচাই করার কমান্ড-লাইন টুল। Deploy-এর আগে ব্যবহার করুন।

WAL (Write-Ahead Log): Prometheus-এ ডেটা লেখার আগে সংরক্ষণ করা log। Crash-এর পর ডেটা উদ্ধার করে।

TSDB: Time Series Database — সময়ের সঙ্গে মান সংরক্ষণের ডেটাবেস। Prometheus-এর storage।

Blind Spot: Monitoring-এর অন্ধ জায়গা যেখানে সমস্যা দেখা যায় না। Platform down হলে blind spot তৈরি হয়।

Service Map: সার্ভিসগুলোর মধ্যে নির্ভরতা ও request প্রবাহের চিত্র। Trace ডেটা থেকে তৈরি করা যায়।

Quick Reference Handbook • Prometheus Reference

Prometheus Reference

prometheus.yml — উদাহরণ

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: technova-prod
    replica: "0"

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

rule_files:
  - "rules/*.yml"

scrape_configs:
  - job_name: "technova-api"
    static_configs:
      - targets: ["api:3000"]
  - job_name: "node"
    static_configs:
      - targets: ["node-exporter:9100"]
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
```

Field ব্যাখ্যা

- `scrape_interval` — কতক্ষণ পরপর metrics টানা হবে। 15s সাধারণ মান।
- `evaluation_interval` — কতক্ষণ পরপর alerting/recording rule মূল্যায়ন হবে।
- `external_labels` — সব metric-এ যুক্ত লেবেল; federation ও HA-তে replica চেনাতে কাজে আসে।
- `alerting.alertmanagers` — Alertmanager-এর ঠিকানা।
- `rule_files` — recording ও alerting rule ফাইলের তালিকা।
- `scrape_configs.job_name` — target গোষ্ঠীর নাম; `job` লেবেল হয়।

Recording ও Alerting Rule উদাহরণ

```
# rules/api.yml
groups:
  - name: api
    interval: 30s
    rules:
      - record: job:http_requests_total:rate5m
        expr: sum by (job) (rate(http_requests_total[5m]))
```

```
      - alert: HighErrorRate
        expr: sum by (job) (rate(http_requests_total{status=~"5.."}[5m])) / sum by (job)
(rate(http_requests_total[5m])) > 0.05
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "High 5xx error rate on {{ $labels.job }}"
          runbook: "https://technova.example/runbooks/high-error-rate"
```

সাধারণ কমান্ড (promtool)

```
promtool check config prometheus.yml
promtool check rules rules/api.yml
promtool query instant http://localhost:9090 'up'
prometheus --config.file=prometheus.yml --storage.tsdb.retention.time=15d
prometheus --config.file=prometheus.yml --storage.tsdb.path=/var/lib/prometheus
```

Retention ও Storage Flags

Flag	অর্থ
`--storage.tsdb.retention.time=15d`	ডেটা কতদিন রাখা হবে (ডিফল্ট 15 দিন)
`--storage.tsdb.retention.size=10GB`	ডেটার সর্বোচ্চ আকার; ছাড়ালে পুরোনো ডেটা মুছে যায়
`--storage.tsdb.path`	ডেটা সংরক্ষণের ডিরেক্টরি
`--web.enable-admin-api`	Admin API চালু করে (সতর্কভাবে ব্যবহার করুন)

সতর্কতা: Retention বাড়ালে disk খরচ বাড়ে। Production-এ disk alert (85% warning, 90% critical) অবশ্যই রাখুন।

Quick Reference Handbook • PromQL Quick Reference

PromQL Quick Reference

Up / Health

```
up == 0
up{job="technova-api"}
count by (job) (up == 0)
absent(up{job="technova-api"})
```

Rate / Traffic

```
rate(http_requests_total[5m])
sum by (service) (rate(http_requests_total[5m]))
increase(http_requests_total[1h])
```

Latency Percentiles

```
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
histogram_quantile(0.99, sum by (le, service)
(rate(http_request_duration_seconds_bucket[5m])))
```

Error Rate

```
sum by (job) (rate(http_requests_total{status=~"5.."}[5m])) / sum by (job)
(rate(http_requests_total[5m]))
sum by (job) (rate(http_requests_total{status=~"5.."}[5m]))
```

Saturation

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
(node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memory_MemTotal_bytes *
100
100 - ((node_filesystem_avail_bytes{mountpoint="/"} /
node_filesystem_size_bytes{mountpoint="/"}) * 100)
```

SLO Burn Rate

```
# 1h burn rate, 99.9% SLO (error budget 0.1%)
sum(rate(http_requests_total{status=~"5.."}[1h])) / sum(rate(http_requests_total[1h])) / 0.001
```

সাধারণ ফাংশন

Function	কাজ
`rate()`	Counter থেকে প্রতি সেকেন্ডে হার
`increase()`	নির্দিষ্ট সময়ে মোট বৃদ্ধি
`irate()`	শেষ দুটি sample দিয়ে তাৎক্ষণিক হার
`histogram_quantile()`	Histogram থেকে percentile
`sum / avg / min / max`	Aggregation
`topk() / bottomk()`	সবচেয়ে বড়/ছোট মানের series
`absent()`	Metric অনুপস্থিত কি না
`count()`	Series-এর সংখ্যা

সাধারণ ভুল

- Counter-এ সরাসরি `avg` করা — `rate()` ছাড়া Counter-এর মান অর্থহীন।
- Range vector খুব ছোট নেওয়া — scrape interval-এর চেয়ে ছোট হলে ডেটা মিস হয়।
- `histogram_quantile`-এ `by (le)` ভুলে যাওয়া — percentile ভুল হয়।
- গড় latency দিয়ে সিদ্ধান্ত নেওয়া — p95/p99 দেখুন।
- High-cardinality লেবেল দিয়ে `by` করা — series বিস্ফোরণ হয়।

Quick Reference Handbook • Grafana Reference

Grafana Reference

Data Source Provisioning YAML

```
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: true
  - name: Loki
    type: loki
    access: proxy
    url: http://loki:3100
  - name: Tempo
    type: tempo
    access: proxy
    url: http://tempo:3200
```

Dashboard Provisioning

```
apiVersion: 1
providers:
  - name: technova
    type: file
    disableDeletion: false
    updateIntervalSeconds: 10
    options:
      path: /etc/grafana/provisioning/dashboards
```

Variables

- `$service` — Query variable: `label_values(http_requests_total, service)`
- `$instance` — Query variable: `label_values(up, instance)`
- `$__rate_interval` — Panel-এর resolution অনুযায়ী স্বয়ংক্রিয় range। `rate()`-এ ব্যবহার করুন।
- `$__interval` — Graph-এর interval। Aggregation-এ ব্যবহার করুন।

Alerting ধারণা

- Grafana alert একাধিক data source-এ query করতে পারে; Prometheus alert শুধু PromQL-এ।
- Alert rule-এ evaluation interval, `for` সময় ও condition ঠিক করুন।
- Contact point (Slack, ইমেইল) ও notification policy দিয়ে routing করুন।
- Silence ও mute timing দিয়ে রক্ষণাবেক্ষণের সময় alert চুপ করান।

উপকারী Panel / Query Pattern

```
# Request rate by service (Time series)
sum by (service) (rate(http_requests_total[$__rate_interval]))
# Error rate % (Stat)
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) * 100
# p95 latency (Time series)
histogram_quantile(0.95, sum by (le)
(rate(http_request_duration_seconds_bucket[$__rate_interval])))
# Log volume (Logs panel, Loki)
{app="api"} |= "error"
```

PRO TIP: Dashboard-এ প্রতিটি panel-এর শিরোনামে একক (unit) লিখুন — যেমন "p95 latency (ms)" — যাতে ভুল ব্যাখ্যা না হয়।

Quick Reference Handbook • Loki & LogQL Reference

Loki & LogQL Reference

Grafana Alloy Config Sketch

```
loki.source.file "api" {
  targets = [{ "__path__" = "/var/log/technova/api.log", "app" = "api" }]
  forward_to = [loki.process.api.receiver]
}
loki.process "api" {
  stage.json {
    expressions = { level = "level", trace_id = "trace_id", msg = "msg" }
  }
  stage.labels {
    values = { level = "level" }
  }
  forward_to = [loki.write.default.receiver]
}
loki.write "default" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
```

LogQL — Log Queries

```
{app="api"}
{app="api", level="error"}
{app="api"} |= "timeout"
{app="api"} != "healthcheck"
{app="api"} |~ "5[0-9]{2}"
{app="api"} |= "error" | json | trace_id="abc123"
```

Label / Line Filter ও Parser

Syntax	অর্থ
`{app="api"}`	Label filter — stream বাছাই
`|= "error"`	Line filter — শব্দ আছে এমন লাইন
`!= "ok"`	Line filter — শব্দ নেই এমন লাইন
`|~ "5[0-9]{2}"`	Regex line filter
`| json`	JSON parser — field বের করে
`| logfmt`	logfmt parser

`| label_format`	লেবেলের মান বদলানো

Metric Queries

```
count_over_time({app="api"} |= "error" [5m])
sum by (level) (count_over_time({app="api"}[5m]))
rate({app="api"} |= "error" [5m])
```

Structured Logging Pattern — JSON Log উদাহরণ

```
{"level":"info","time":"2026-11-09T10:00:00Z","service":"api","trace_id":"4bf92f3577b34da6a3ce
929d0e0e4736","request_id":"req-9f2c41","method":"GET","path":"/api/orders","status":
200,"duration_ms":42,"msg":"request completed"}
{"level":"error","time":"2026-11-09T10:01:12Z","service":"api","trace_id":"4bf92f3577b34da6a3c
e929d0e0e4736","request_id":"req-9f2c42","method":"GET","path":"/api/orders","status":
500,"duration_ms":1200,"msg":"DB timeout"}
```

নিয়ম: প্রতিটি log-এ `level`, `trace_id`, `request_id` ও `service` রাখুন। পাসওয়ার্ড, টোকেন বা ব্যক্তিগত তথ্য কখনো log-এ লিখবেন না।

Quick Reference Handbook • OpenTelemetry Reference

OpenTelemetry Reference

Collector YAML — Receivers / Processors / Exporters / Pipelines

```
receivers:
  otlp:
    protocols:
      http:
        endpoint: 0.0.0.0:4318
      grpc:
        endpoint: 0.0.0.0:4317

processors:
  batch:
    timeout: 5s
  tail_sampling:
    decision_wait: 10s
    policies:
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }

exporters:
  otlp/tempo:
    endpoint: tempo:4317
    tls: { insecure: true }
  prometheus:
    endpoint: 0.0.0.0:8889

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [tail_sampling, batch]
      exporters: [otlp/tempo]
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [prometheus]
```

Node.js Auto-Instrumentation Snippet

```
// instrumentation.js — অ্যাপের আগে লোড করুন
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');
const { OTLPTraceExporter } = require('@opentelemetry/exporter-trace-otlp-http');

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({ url: 'http://localhost:4318/v1/traces' }),
  instrumentations: [getNodeAutoInstrumentations()],
```

```
});
sdk.start();
```

```
node --require ./instrumentation.js app.js
```

Trace Investigation ধাপ

1. Grafana-তে trace ID দিয়ে Tempo-তে trace খুলুন।
2. সবচেয়ে দীর্ঘ span খুঁজুন — সেখানেই সময় ব্যয় হয়েছে।
3. Span-এর attribute (যেমন `db.statement`, `http.status_code`) দেখুন।
4. Error span-এর event ও log মিলিয়ে দেখুন।
5. একই trace ID দিয়ে Loki-তে log খুঁজুন।

Sampling Config

```
# Head sampling: 10% trace রাখুন
# SDK-তে: tracesSampler: TraceIdRatioBasedSampler(0.1)
# Tail sampling: Collector-এ error ও ধীর trace রাখুন
# decision_wait: 10s, policy: status_code ERROR বা latency > 1s
```

সতর্কতা: Sampling-এ বিরল error trace হারাতে পারে। Error trace-এর জন্য tail sampling-এ অগ্রাধিকার দিন।

Quick Reference Handbook • Kubernetes Monitoring Commands

Kubernetes Monitoring Commands

```
kubectl top node
kubectl top pod -n technova
kubectl top pod --sort-by=cpu -n technova
kubectl get pods -n technova
kubectl get pods -n technova -o wide
kubectl describe pod api-7d9f8c6b5-x2k4p -n technova
kubectl logs api-7d9f8c6b5-x2k4p -n technova
kubectl logs -f api-7d9f8c6b5-x2k4p -n technova
kubectl logs --previous api-7d9f8c6b5-x2k4p -n technova
kubectl get events -n technova --sort-by=.lastTimestamp
kubectl get events --all-namespaces --sort-by=.lastTimestamp | tail -20
kubectl get deployments -n technova
kubectl rollout status deployment/api -n technova
kubectl get svc -n technova
kubectl get nodes -o wide
```

kube-state-metrics নোট

- kube-state-metrics Kubernetes API-র অবস্থাকে metrics-এ রূপান্তর করে: `kube_pod_status_phase`, `kube_deployment_status_replicas_available`, `kube_pod_container_status_restarts_total`।
- metrics-server শুধু বর্তমান CPU/memory দেয়; kube-state-metrics অবস্থা দেয়। দুটি আলাদা কাজ।
- Prometheus-এ Kubernetes service discovery দিয়ে kube-state-metrics scrape করুন।

সাধারণ Pod / Container Metric Query

```
rate(container_cpu_usage_seconds_total{pod="api-7d9f8c6b5-x2k4p"}[5m])
container_memory_usage_bytes{pod="api-7d9f8c6b5-x2k4p"} / container_spec_memory_limit_bytes
sum by (pod) (rate(container_cpu_usage_seconds_total{namespace="technova"}[5m]))
kube_pod_container_status_restarts_total{namespace="technova"}
kube_deployment_status_replicas_available{deployment="api"} /
kube_deployment_spec_replicas{deployment="api"}
```

Quick Reference Handbook • Checklists

Checklists

Production Observability Checklist

- ☐ Prometheus সব target scrape করছে (`up==1`) এবং scrape interval ঠিক আছে।
- ☐ Node Exporter সব node-এ চলছে; kube-state-metrics ও metrics-server চালু আছে।
- ☐ Golden Signals dashboard আছে: Latency, Traffic, Errors, Saturation।
- ☐ Alert actionable, severity ঠিক, runbook link আছে, `for` সময় দেওয়া আছে।
- ☐ Alertmanager-এ routing, receiver ও silence পরীক্ষা করা হয়েছে।
- ☐ Loki-তে structured log আসছে; প্রতিটি log-এ `trace_id` ও `request_id` আছে।
- ☐ Trace context propagation সব সার্ভিসে কাজ করছে; Tempo-তে trace দেখা যায়।
- ☐ SLI/SLO ঠিক করা আছে; error budget dashboard আছে।
- ☐ Retention, disk alert ও capacity পরিকল্পনা আছে।
- ☐ Observability platform-এর নিজের health monitoring আছে (blind spot নেই)।
- ☐ Backup ও restore পরীক্ষা করা হয়েছে।

Security & Privacy Checklist

- ☐ Log, metric label বা trace-এ পাসওয়ার্ড, টোকেন বা API key নেই।
- ☐ Log-এ PII (নাম, ইমেইল, ফোন) redact/mask করা হয়েছে।
- ☐ Grafana, Prometheus ও Loki-তে authentication ও access control চালু আছে।
- ☐ Role অনুযায়ী access দেওয়া হয়েছে (viewer/editor/admin)।
- ☐ Secret শুধু secure vault বা environment variable-এ আছে, config ফাইলে নয়।
- ☐ TLS certificate-এর মেয়াদ monitoring করা হচ্ছে।
- ☐ Audit log চালু আছে — কে কী দেখেছে তা জানা যায়।

SLI / SLO Reference

30 দিন = 43,200 মিনিট। Error budget = (100% − SLO) × 43,200 মিনিট।

SLO	Error Budget	অনুমোদিত Downtime / মাস (30 দিন)
99.9%	0.1% = 43.2 মিনিট	43 মিনিট 12 সেকেন্ড
99.95%	0.05% = 21.6 মিনিট	21 মিনিট 36 সেকেন্ড
99.99%	0.01% = 4.32 মিনিট	4 মিনিট 19 সেকেন্ড

গাণিতিক যাচাই নোট: 30 দিনের মাসে 99.99%-এর সঠিক মান 4 মিনিট 19 সেকেন্ড (4.32 মিনিট)। 4 মিনিট 23 সেকেন্ড মানটি প্রায় 30.44 দিনের (গড়) মাসের জন্য প্রযোজ্য, 30 দিনের মাসের জন্য নয় — তাই উপরের টেবিলে 30 দিনের হিসাবে 4 মিনিট 19 সেকেন্ড লেখা হয়েছে।

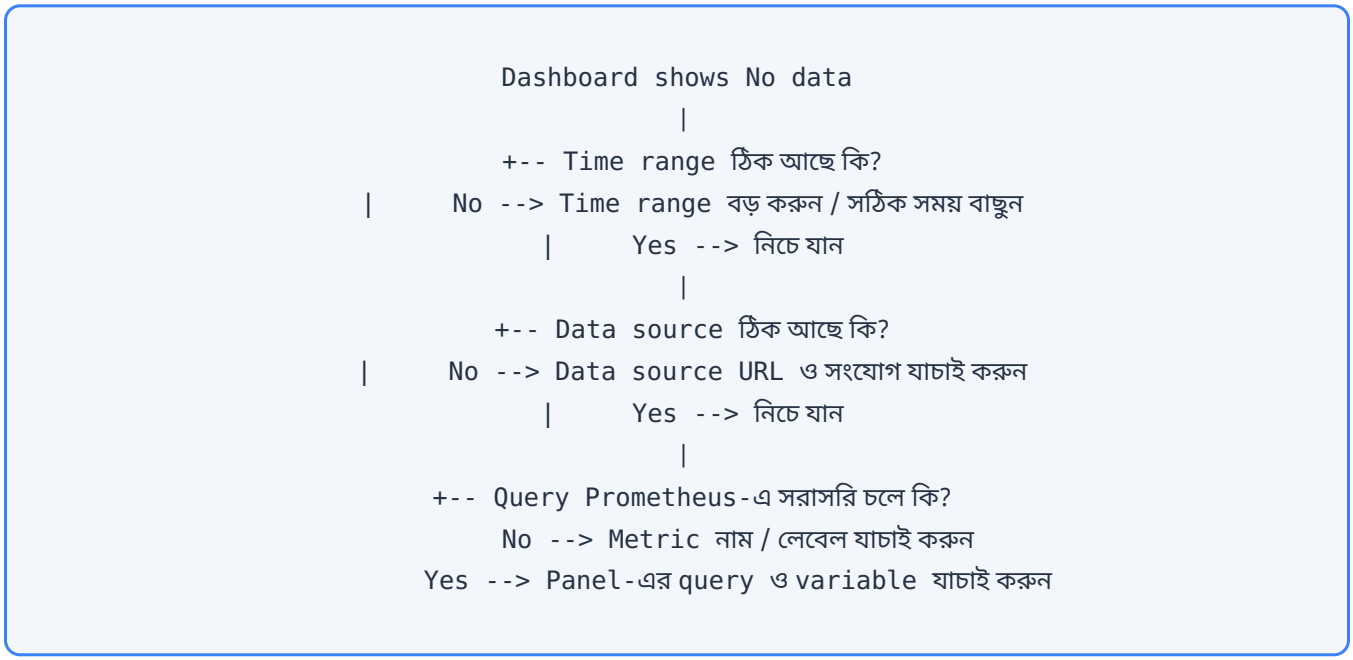
Incident Investigation Checklist

1. প্রভাব নিশ্চিত করুন: কতজন ব্যবহারকারী, কোন সার্ভিস, কখন শুরু।
2. যোগাযোগ শুরু করুন: incident channel-এ আপডেট দিন।
3. Golden Signals দেখুন: Latency, Traffic, Errors, Saturation-এ কী বদলেছে।
4. সময় মিলান: deploy, config change বা downstream incident-এর সঙ্গে মিল আছে কি না।
5. Log দেখুন: error log-এর প্রথম ঘটনা কখন, কোন service-এ।
6. Trace দেখুন: ধীর span কোথায়, কোন downstream-এ।
7. Mitigation করুন: rollback, traffic সরানো বা scale করা।
8. Root cause লিখুন: প্রমাণসহ, অনুমান আলাদা করে।
9. Postmortem লিখুন: timeline, শিক্ষা ও প্রতিরোধমূলক পদক্ষেপ।

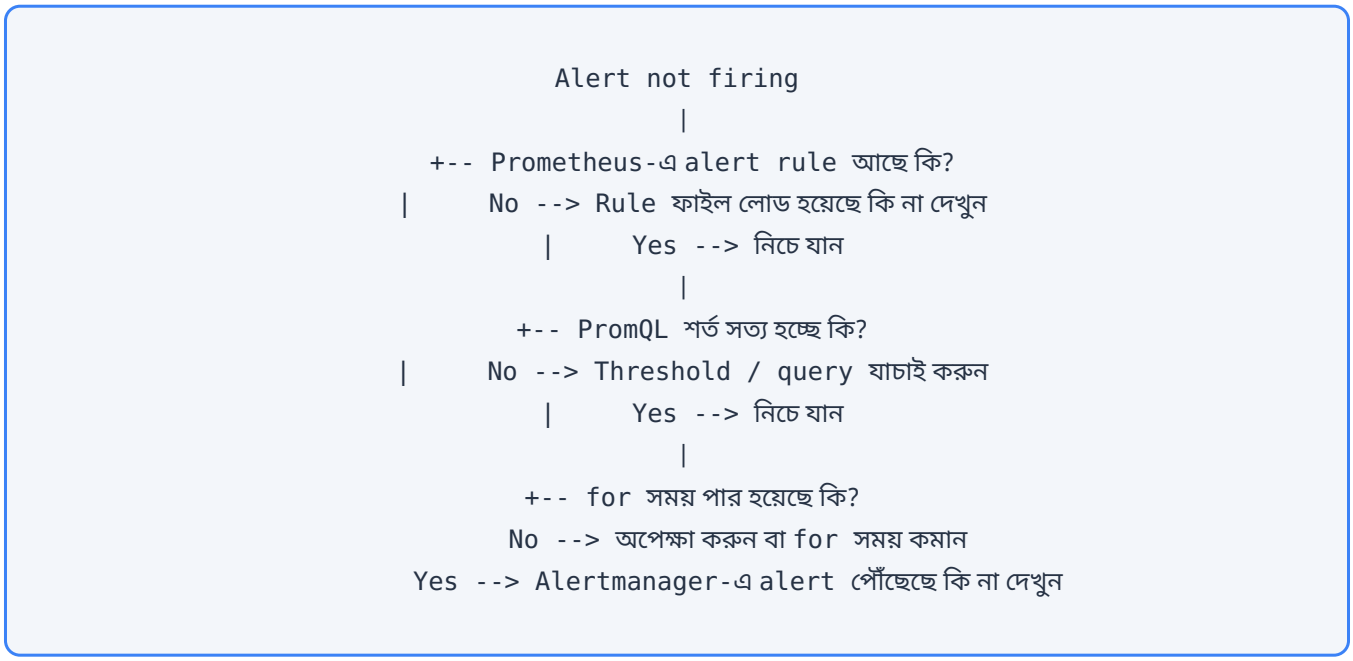
Quick Reference Handbook • Troubleshooting Decision Trees

Troubleshooting Decision Trees

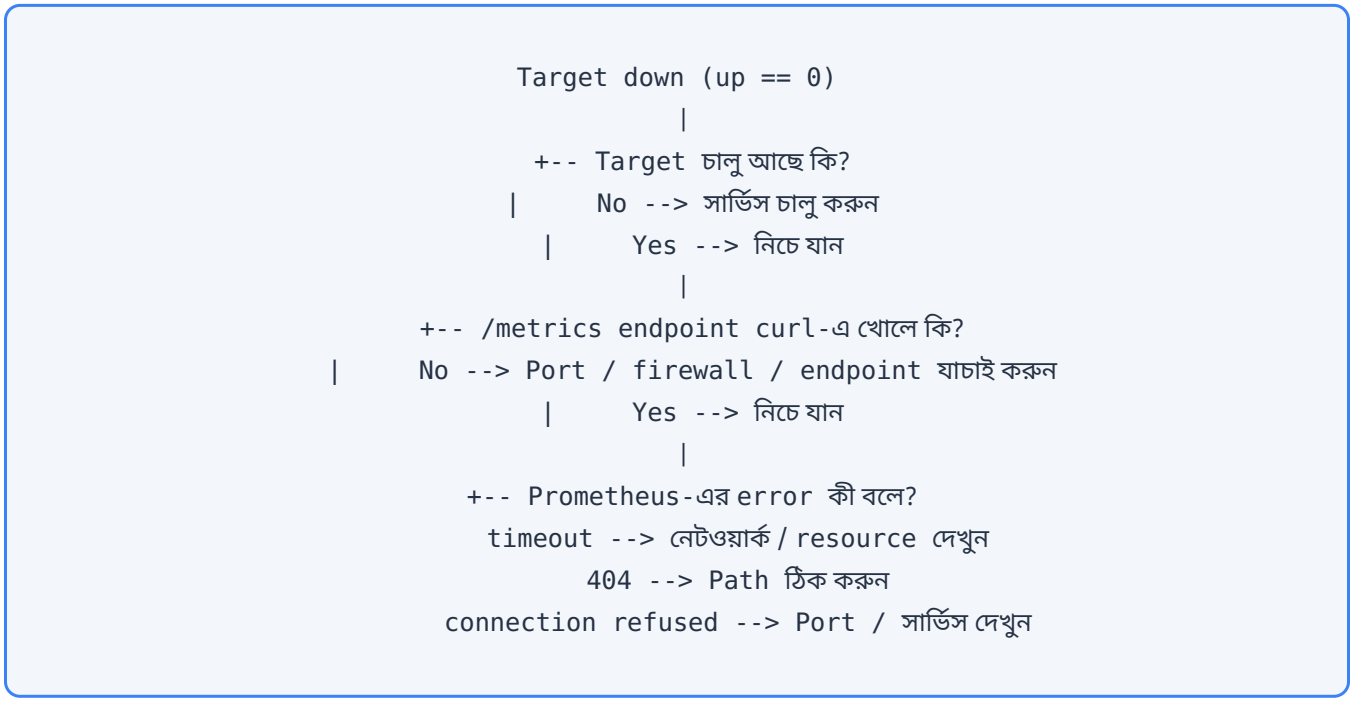
Dashboard Shows No Data



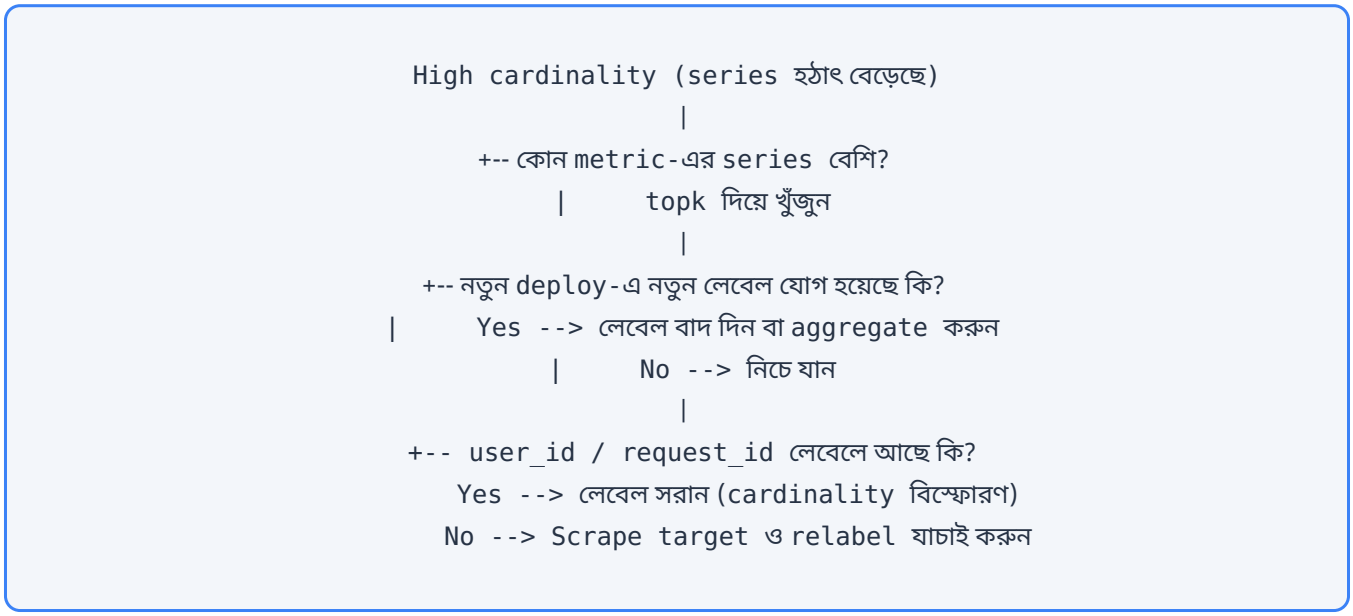
Alert Not Firing



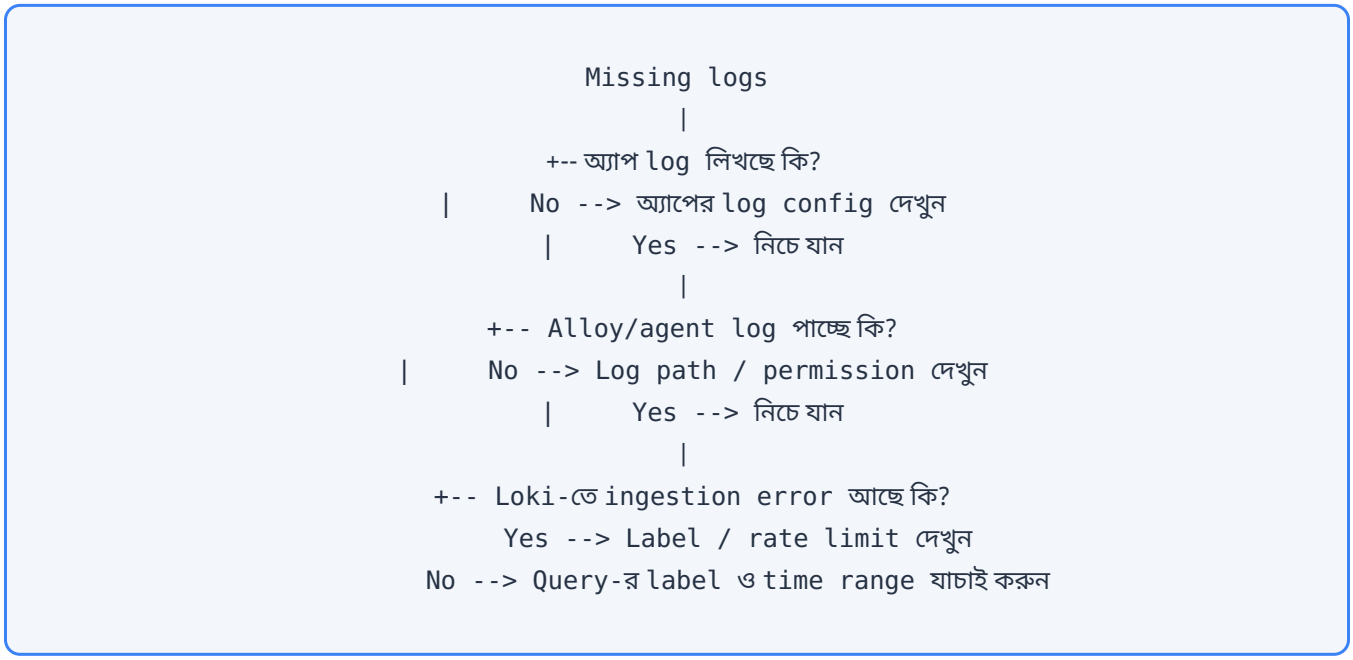
Target Down



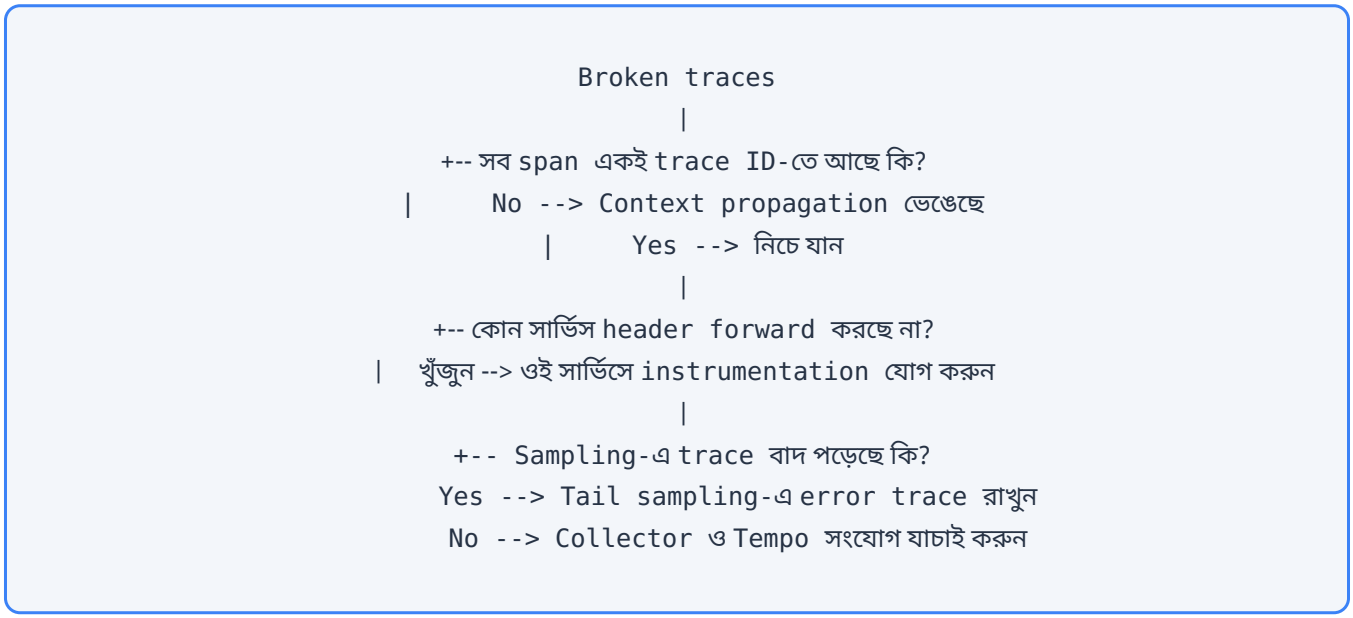
High Cardinality



Missing Logs



Broken Traces



Quick Reference Handbook • Personal Competency Tracking Sheet

Personal Competency Tracking Sheet

প্রতিটি দক্ষতায় নিজেকে যাচাই করুন: **Can explain** (বোঝাতে পারি), **Can do in lab** (ল্যাবে করতে পারি), **Can do in production** (প্রোডাকশনে করতে পারি)। তারিখ লিখে অগ্রগতি ধরে রাখুন।

Skill	Can explain	Can do in lab	Can do in production	Date
Monitoring vs Observability ব্যাখ্যা করা				
Metrics, Logs, Traces-এর পার্থক্য ও ব্যবহার				
Counter, Gauge, Histogram, Summary বাছাই করা				
Prometheus ইনস্টল ও configure করা				
PromQL query লেখা (rate, histogram_quantile)				
Exporter ও Node Exporter ব্যবহার করা				
Recording ও Alerting Rule লেখা				
Alertmanager routing, grouping, silencing				
Kubernetes monitoring (kube-state-metrics, kubectl top)				
Grafana dashboard তৈরি করা				
Grafana variable, provisioning ও alerting				
Structured logging (JSON log) লেখা				
Loki ও LogQL query লেখা				
Elastic / OpenSearch-এর সঙ্গে Loki-র তুলনা				
OpenTelemetry instrumentation করা				
OTel Collector configure করা				
Jaeger / Tempo-তে trace বিশ্লেষণ করা				
Metrics, logs, traces correlate করা				
APM ও profiling ব্যবহার করা				
SLI, SLO, Error Budget ঠিক করা				
Burn rate alert লেখা				
Cardinality নিয়ন্ত্রণ করা				

Sampling নীতি ঠিক করা				
Retention ও cost নিয়ন্ত্রণ করা				
Log-এ PII / secret রোধ করা				
Prometheus high availability নকশা করা				
Incident তদন্ত ও mitigation করা				
Blameless postmortem লেখা				

Quick Reference Handbook • Official References

Official References

নিচের তালিকায় শুধু official documentation-এর আসল base URL দেওয়া হয়েছে। বিস্তারিত জানতে সংশ্লিষ্ট সাইটের documentation বিভাগ দেখুন।

- Prometheus Documentation — prometheus.io
- PromQL Documentation — prometheus.io
- Grafana Documentation — grafana.com
- Loki Documentation — grafana.com
- Grafana Alloy Documentation — grafana.com
- OpenTelemetry Documentation — opentelemetry.io
- Jaeger Documentation — jaegertracing.io
- Tempo Documentation — grafana.com
- Elasticsearch Documentation — elastic.co
- OpenSearch Documentation — opensearch.org
- Kubernetes Documentation — kubernetes.io
- CNCF Resources — cncf.io

সতর্কতা: উপরের base domain ছাড়া কোনো deep link এখানে দেওয়া হয়নি। কোনো URL-এ সন্দেহ হলে official সাইটের search ব্যবহার করুন। কাল্পনিক কোম্পানি TechNova, ডোমেইন `technova.example` ও IP `203.0.113.x` শুধু উদাহরণের জন্য।